

SIMULATING AND ANALYZING PROCESS MANAGEMENT IN AN DOCUMENTATION

Project Title: SIMULATING AND ANALYZING PROCESS MANAGEMENT IN AN

Student Name: Hazel Arana

Course & Section: BSCS – 2

Date: March 14, 2026

Process Illustration (Diagram)

Process States & Transitions:

1. **New:** The process is being created.
 - *Transition:* Admitted -> **Ready**
2. **Ready:** The process is waiting to be assigned to a processor.
 - *Transition:* Scheduler Dispatch -> **Running**
3. **Running:** Instructions are being executed by the CPU.
 - *Transition 1:* Interrupt / Time Slice Expired -> **Ready**
 - *Transition 2:* I/O or Event Wait -> **Waiting**
 - *Transition 3:* Exit -> **Terminated**
4. **Waiting (Blocked):** The process is waiting for some event to occur (like an I/O completion).
 - *Transition:* I/O or Event Completion -> **Ready**
5. **Terminated:** The process has finished execution.

Role of the Process Control Block (PCB): The PCB is a data structure used by the OS to store all the information about a specific process (like its current state, Program Counter, CPU registers, and CPU scheduling info). It acts as the "brain" or "bookmark" for a process. When the CPU switches from one process to another (Context Switch), it saves the current process's data into its PCB and loads the next process's data from its PCB so it can resume exactly where it left off.

CPU Scheduling Simulation

Given Data:

- **P1:** Arrival = 0 ms | Burst = 5 ms

- **P2:** Arrival = 1 ms | Burst = 3 ms
- **P3:** Arrival = 2 ms | Burst = 8 ms
- **P4:** Arrival = 3 ms | Burst = 6 ms

Formulas used:

- **Turnaround Time (TAT)** = Completion Time - Arrival Time
- **Waiting Time (WT)** = Turnaround Time - Burst Time

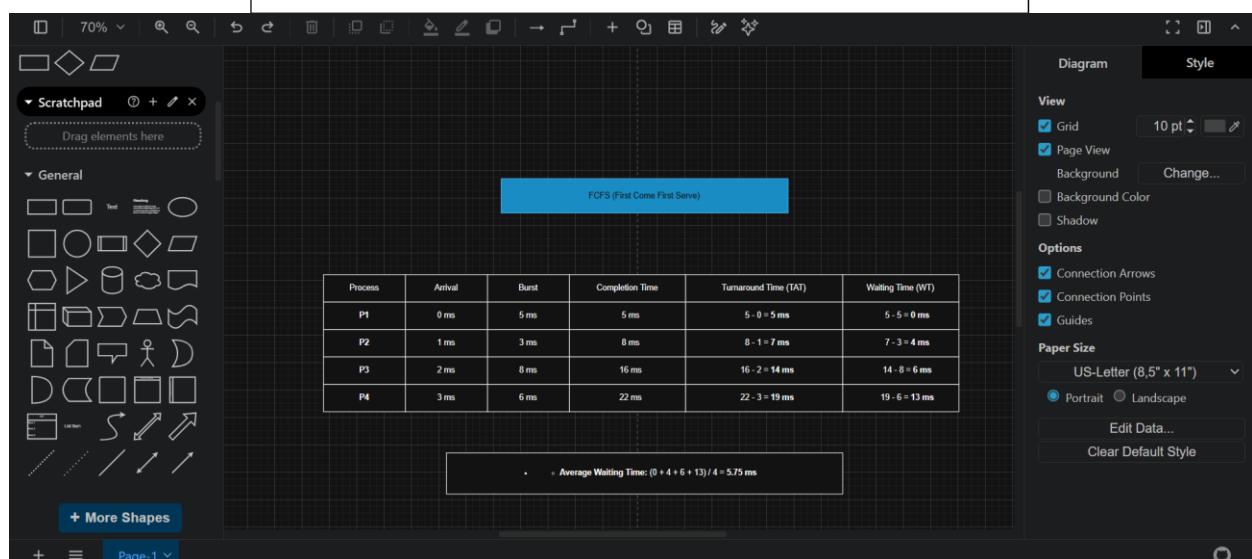
1. FCFS (First Come First Serve)

Processes are executed strictly in the order they arrive.

FCFS (First Come First Serve)

Process	Arrival	Burst	Completion Time	Turnaround Time (TAT)	Waiting Time (WT)
P1	0 ms	5 ms	5 ms	$5 - 0 = 5 \text{ ms}$	$5 - 5 = 0 \text{ ms}$
P2	1 ms	3 ms	8 ms	$8 - 1 = 7 \text{ ms}$	$7 - 3 = 4 \text{ ms}$
P3	2 ms	8 ms	16 ms	$16 - 2 = 14 \text{ ms}$	$14 - 8 = 6 \text{ ms}$
P4	3 ms	6 ms	22 ms	$22 - 3 = 19 \text{ ms}$	$19 - 6 = 13 \text{ ms}$

- ○ Average Waiting Time: $(0 + 4 + 6 + 13) / 4 = 5.75 \text{ ms}$



2. SJF (Shortest Job First) - Non-Preemptive

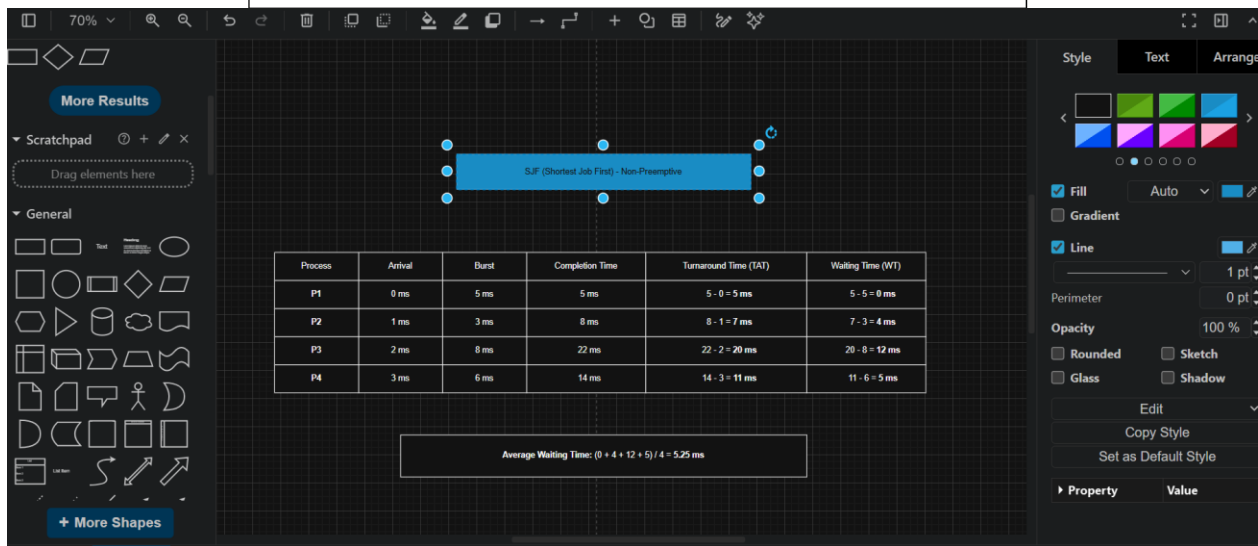
At the time of scheduling, the CPU picks the available process with the shortest burst time.

Gantt Chart: (Note: At $T=0$, only $P1$ is available, so it runs first. By $T=5$, $P2$, $P3$, and $P4$ have arrived. $P2$ is the shortest, then $P4$, then $P3$).

SJF (Shortest Job First) - Non-Preemptive

Process	Arrival	Burst	Completion Time	Turnaround Time (TAT)	Waiting Time (WT)
P1	0 ms	5 ms	5 ms	$5 - 0 = 5$ ms	$5 - 5 = 0$ ms
P2	1 ms	3 ms	8 ms	$8 - 1 = 7$ ms	$7 - 3 = 4$ ms
P3	2 ms	8 ms	22 ms	$22 - 2 = 20$ ms	$20 - 8 = 12$ ms
P4	3 ms	6 ms	14 ms	$14 - 3 = 11$ ms	$11 - 6 = 5$ ms

$$\text{Average Waiting Time: } (0 + 4 + 12 + 5) / 4 = 5.25 \text{ ms}$$



3. Round Robin (Time Quantum = 3 ms)

Processes take turns running for a maximum of 3 ms. If they aren't finished, they go to the back of the Ready Queue.

(Execution breakdown: P1 runs for 3ms. P2 runs for 3ms and finishes. P3 runs for 3ms. P4 runs for 3ms. P1 runs for its remaining 2ms and finishes. P3 runs for 3ms. P4 runs for its remaining 3ms and finishes. P3 runs for its remaining 2ms and finishes).

SJF (Shortest Job First) - Non-Preemptive

Process	Arrival	Burst	Completion Time	Turnaround Time (TAT)	Waiting Time (WT)
P1	0 ms	5 ms	5 ms	$5 - 0 = 5 \text{ ms}$	$5 - 5 = 0 \text{ ms}$
P2	1 ms	3 ms	8 ms	$8 - 1 = 7 \text{ ms}$	$7 - 3 = 4 \text{ ms}$
P3	2 ms	8 ms	22 ms	$22 - 2 = 20 \text{ ms}$	$20 - 8 = 12 \text{ ms}$
P4	3 ms	6 ms	14 ms	$14 - 3 = 11 \text{ ms}$	$11 - 6 = 5 \text{ ms}$

Average Waiting Time: $(0 + 4 + 12 + 5) / 4 = 5.25 \text{ ms}$

More Results

Scratchpad

Drag elements here

General

Process

Process	Arrival	Burst	Completion Time	Turnaround Time (TAT)	Waiting Time (WT)
P1	0 ms	5 ms	5 ms	$5 - 0 = 5 \text{ ms}$	$5 - 5 = 0 \text{ ms}$
P2	1 ms	3 ms	8 ms	$8 - 1 = 7 \text{ ms}$	$7 - 3 = 4 \text{ ms}$
P3	2 ms	8 ms	22 ms	$22 - 2 = 20 \text{ ms}$	$20 - 8 = 12 \text{ ms}$
P4	3 ms	6 ms	14 ms	$14 - 3 = 11 \text{ ms}$	$11 - 6 = 5 \text{ ms}$

Average Waiting Time: $(0 + 4 + 12 + 5) / 4 = 5.25 \text{ ms}$

Style

Text

Arrange

Fill

Auto

Line

1 pt

Perimeter

0 pt

Opacity

100 %

Rounded

Sketch

Glass

Shadow

Edit

Copy Style

Set as Default Style

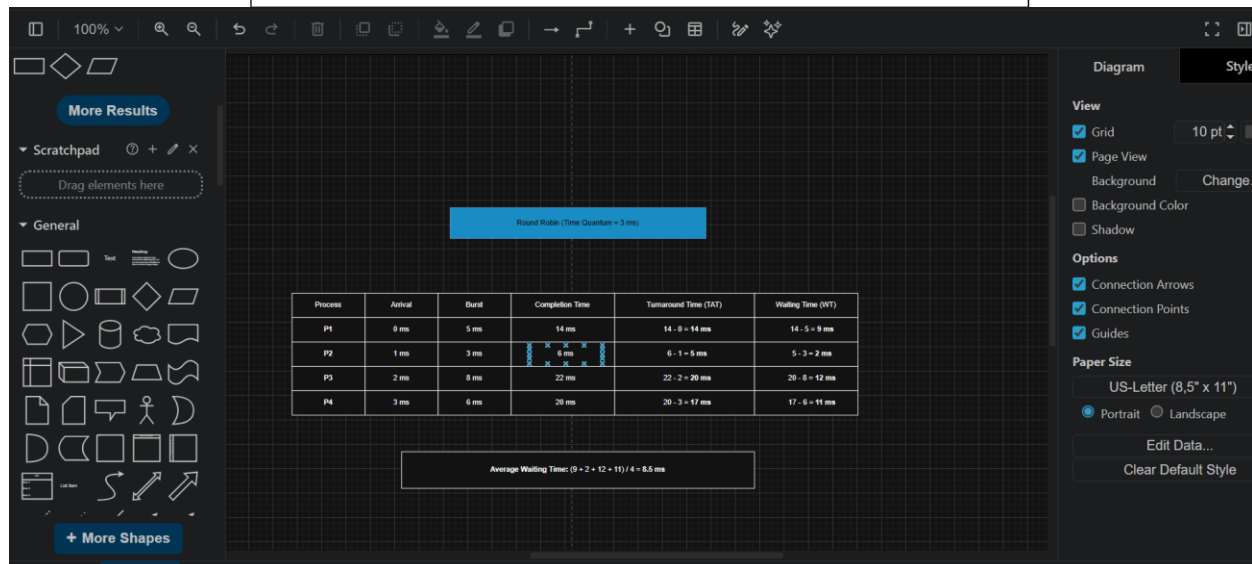
Property

Value

Round Robin (Time Quantum = 3 ms)

Process	Arrival	Burst	Completion Time	Turnaround Time (TAT)	Waiting Time (WT)
P1	0 ms	5 ms	14 ms	$14 - 0 = 14 \text{ ms}$	$14 - 5 = 9 \text{ ms}$
P2	1 ms	3 ms	6 ms	$6 - 1 = 5 \text{ ms}$	$5 - 3 = 2 \text{ ms}$
P3	2 ms	8 ms	22 ms	$22 - 2 = 20 \text{ ms}$	$20 - 8 = 12 \text{ ms}$
P4	3 ms	6 ms	20 ms	$20 - 3 = 17 \text{ ms}$	$17 - 6 = 11 \text{ ms}$

Average Waiting Time: $(9 + 2 + 12 + 11) / 4 = 8.5 \text{ ms}$



Reflection and Analysis

1. Which scheduling algorithm performed best? Why? Based on the calculations, **Shortest Job First (SJF)** performed the best because it yielded the lowest Average Waiting Time (**5.25 ms**), compared to FCFS (5.75 ms) and Round Robin (8.5 ms). By getting the shorter processes out of the way quickly, SJF minimizes the time that short processes spend waiting behind long ones, which mathematically guarantees the minimum average waiting time for a given set of processes.

2. How does PCB help the OS manage processes? The Process Control Block (PCB) acts as the fundamental tracking mechanism for the OS. In preemptive algorithms like Round Robin, the CPU frequently interrupts running processes. The PCB helps by saving the exact state (register values, program counter, and memory limits) of the interrupted process. When it is that process's turn to run again, the OS reads the PCB to resume execution exactly where it left off, ensuring multiple processes can run concurrently without corrupting each other's data.

3. What problems occur if process management is poor? If process management is poorly implemented, several critical issues can arise:

- **Starvation:** Low-priority or long processes may never get CPU time if shorter or higher-priority processes keep arriving.
- **Deadlock:** Two or more processes might end up waiting indefinitely for resources held by each other, causing the system to freeze.
- **High Latency/Poor Responsiveness:** An inefficient scheduler (like FCFS handling a massive burst-time process first) creates the "Convoy Effect," leading to a sluggish user experience and poor overall CPU utilization.